The BJKST Algorithm for Counting Distinct Elements in Data Streams

Theoretical foundations, hash-function blackboxes, Markov bounds, and a verified cricket-stream case study

Randomized & Streaming Algorithms — Corrected Project Report | October 2026

Abstract

Estimating the number of distinct items in a stream (the zeroth frequency moment, F_0) is fundamental to streaming analytics. This report explains the BJKST adaptive-subsampling algorithm without implementation code. It treats hashing as a statistical blackbox, develops an illustrative stream trace, and proves a constant-success relative-error bound using Markov's inequality, Chebyshev's inequality, and a union bound over adaptive sampling levels. Exact cardinalities are independently recalculated from the supplied 60,000-row synthetic cricket dataset. Previously reported randomized estimates are not presented as verified experiments because their hash seeds and execution records are unavailable.

1. Problem Scenario: Distinct Counting in a Stream

Let $S = (a_1, \ldots, a_m)$ be a stream of m arriving identifiers drawn from a universe U of size n. The distinct count F_0 is the number of different identifiers occurring at least once. An exact set-based method stores each observed unique identifier; its conventional representation can require $O(F_0 \log n)$ bits, whereas a universe bitmap requires n bits. For large domains or composite identifiers, maintaining an exact set can be costly. BJKST instead estimates the count using a bounded buffer and a randomized hash function.

1.1 Verified dataset scenario

The supplied cricket_deliveries.csv contains 60,000 synthetic ball-by-ball rows. Each row has delivery_id, batting_team, batter, bowler, venue, runs, and dismissal. The table below reports exact distinct counts recalculated directly from the CSV, not estimates.

Stream key	Exact distinct F0
Delivery identifiers	60,000
Batting teams	10
Venues	25
Bowlers	150
Batters	200
Batter-bowler pairs	16,076
Batter-bowler-venue-runs tuples	54,397

Composite keys can have much higher cardinality than individual columns. The algorithm can process the identifier from each arriving row and estimate the number of unique identifiers or tuples without retaining every past tuple.

2. Hash Functions as Blackboxes

Choose a hash function h from a pairwise-independent family mapping U to L-bit strings. We use only the following statistical properties, not the internal construction of the hash function.

Uniformity. For every fixed item u, h(u) is uniform over the 2^L outputs. Writing r(u) for its number of trailing binary zeros, $P(r(u) \geq z) = 2^{-z}$ for $0 \leq z \leq L$.

Pairwise independence. For distinct items u and v, the random values h(u) and h(v) are independent. Their survival indicators therefore have zero covariance, enabling a variance bound.

Consistency. Once h is selected, repeated occurrences of the same item produce the same output and the same sampling decision. Duplicates do not increase the number of distinct items retained.

Collisions. For an output range of size $M = 2^L$, a union bound gives collision probability at most $F_0(F_0-1)/(2M)$ among the distinct stream items. A sufficiently large range makes this small; the sample buffer stores identifiers as well as their trailing-zero counts, so equal hash outputs need not be mistaken for identical identifiers.

3. BJKST Technique: Adaptive Subsampling

Maintain an integer sampling level $z \geq 0$ and a set B of distinct retained identifiers, each associated with its trailing-zero count r. Let k be the buffer capacity. Initially $z = 0$ and B is empty. When an element arrives, compute r from the hash blackbox. Ignore it if $r < z$; otherwise insert it if not already present. If the buffer exceeds k, increase z by one and evict every buffered item with $r < z$. Repeat if needed. Increasing z halves the survival probability at each step. The final estimate is:

$$\hat{F}0 = |B| \times 2^z$$

For the conservative proof in Section 5, choose $k = \text{ceil}(144 / \varepsilon^2)$ for $0 < \varepsilon < 1$. This is a sufficient proof constant, not an optimized one. The much smaller capacity $k = 3$ below is purely pedagogical and does not carry that guarantee.

4. Worked Example (Illustrative Hash Assignments)

The following 12-token sequence and six-bit hash outputs reproduce the explanatory example in the original report. They are illustrative, not the first 12 rows of the uploaded CSV. The actual first 12 bowler identifiers differ from this sequence.

Stream: B140, B042, B138, B005, B003, B140, B042, B019, B088, B112, B005, B077. There are 9 distinct identifiers.

Item	Hash output	Trailing zeros
B140	101100	2
B042	011010	1
B138	100111	0
B005	111001	0
B003	010100	2
B019	010110	1

Item	Hash output	Trailing zeros
B088	110101	0
B112	001100	2
B077	101010	1

4.1 Buffer trace (k = 3)

Arrival(s)	Action	Level z	Buffer after action
1–3: B140, B042, B138	Insert three distinct items	0	B140, B042, B138
4: B005	Overflow; increase z and evict r = 0	1	B140, B042
5: B003	Insert	1	B140, B042, B003
6–7: repeats	Duplicates ignored	1	B140, B042, B003
8: B019	Overflow; increase z and evict r = 1	2	B140, B003
9: B088	Rejected (r = 0)	2	B140, B003
10: B112	Insert	2	B140, B003, B112
11–12: B005, B077	Rejected (r < 2)	2	B140, B003, B112

$$\textbf{Estimate} = 3 \times 2^2 = 12; \text{ exact } F0 = 9; \text{ relative error} = 33.33\%.$$

The intentionally tiny sample makes level changes visible but also produces a substantial estimation error. This does not contradict the theoretical guarantee, which requires a capacity chosen for the desired ε.

5. Mathematical Analysis: Markov’s Inequality

5.1 Survivor count at a fixed level

Let D be the set of distinct stream identifiers, $|D| = F_0$. For each u in D, let $X_{u,z}$ equal 1 if $r(u) \geq z$ and 0 otherwise. Define $Y_z = \Sigma_{u \text{ in } D} X_{u,z}$. Uniformity gives $E[X_{u,z}] = 2^{-z}$, hence:

$$\mu_z = E[Y_z] = F_0 / 2^z.$$

At each fixed z, $2^z Y_z$ is unbiased for F_0. Since the final level is selected adaptively, a fixed-level concentration bound cannot be applied to it without an additional argument.

5.2 Markov bound for overshooting the target level

Markov’s inequality says $P(Y \geq a) \leq E[Y]/a$ for nonnegative Y and $a > 0$. If $F_0 \leq k$, the algorithm retains all distinct items and counts exactly. Otherwise choose z* as the smallest integer satisfying $\mu_{z^*} < k/6$. The geometric halving of μ gives $k/12 \leq \mu_{z^*} < k/6$. To increase the final level beyond z*, the number of survivors at z* must exceed k. Thus:

$$P(z_{\text{final}} > z^*) \leq P(Y_{z^*} \geq k) \leq \mu_{z^*}/k < 1/6.$$

5.3 Markov applied to squared deviation

Pairwise independence implies $\mathrm{Var}(Y_z) = \Sigma\, \mathrm{Var}(X_{u,z}) \le \mu_z$. Apply Markov's inequality to the nonnegative squared deviation $W = (Y_z - \mu_z)^2$. For any fixed z:

$$\mathbf{P(|Y_z - \mu_z| \ge \varepsilon\, \mu_z) \le 1/(\varepsilon^2\, \mu_z).}$$

This is the Chebyshev bound obtained directly from Markov's inequality.

5.4 Accounting for the adaptive final level

Use a union bound over all fixed levels 0, 1, ..., z*. Because $\mu_z = F_0/2^z$, the inverse expectations form a geometric series:

$$\mathbf{P(\text{any bad level } z \le z^*) \le \Sigma\, 1/(\varepsilon^2\, \mu_z) < 2/(\varepsilon^2\, \mu_{z^*}) \le 24/(\varepsilon^2 k).}$$

With $k = \mathrm{ceil}(144/\varepsilon^2)$, this is at most 1/6. The overflow event is also less than 1/6, so by the union bound:

$$\mathbf{P(|\hat{F}0 - F_0| \ge \varepsilon\, F_0) \le 1/3.}$$

Consequently a single instance succeeds with probability at least 2/3. Running O(log(1/δ)) independent instances and returning their median amplifies the success probability to at least $1 - \delta$. This proof explicitly handles the random final level rather than substituting it into a fixed-level variance bound.

6. Dataset Evaluation and Interpretation

The following exact cardinalities were computed from the uploaded CSV. They supersede the inconsistent exact counts in the original report. Randomized BJKST estimates previously listed in that report cannot be independently validated without the hash family, random seed, and run record; they are therefore excluded from this verified-results table.

Target stream	Exact F0	Result with k = 512
Bowler IDs	150	Exact: 150; z = 0
Batter IDs	200	Exact: 200; z = 0
Batter-bowler pairs	16,076	Randomized estimate not verified
Batter-bowler-venue-runs tuples	54,397	Randomized estimate not verified

When $F_0 \le 512$, the algorithm never overflows and its estimate is exact. For higher-cardinality composite keys, a bounded buffer may retain far fewer identifiers than an exact set, but actual memory usage in bytes depends on implementation overhead. A reduction in retained element count must not be presented as a measured memory reduction.

To report reproducible randomized results in future work, document the hash family and range, random seed, stream order, buffer capacity, final level, final buffer size, and repeated-run error distribution. The illustrative hash trace in Section 4 should not be confused with such an experiment.

7. Conclusion

BJKST estimates distinct elements in one pass by combining consistent randomized hashing with adaptive subsampling. Uniformity gives the expected sample size; pairwise independence controls its variance. Markov's inequality limits excessive level increases and, applied to squared deviation, yields a concentration bound. A union bound over fixed levels accounts for adaptive selection. The verified cricket dataset illustrates why composite streaming keys can have high cardinality, while the worked trace shows the mechanics without requiring any source code.

References

[1] Z. Bar-Yossef, T. S. Jayram, R. Kumar, D. Sivakumar, and L. Trevisan, "Counting distinct elements in a data stream," RANDOM, 2002.

[2] N. Alon, Y. Matias, and M. Szegedy, "The space complexity of approximating the frequency moments," Journal of Computer and System Sciences 58(1), 1999.

[3] P. Flajolet and G. N. Martin, "Probabilistic counting algorithms for data base applications," Journal of Computer and System Sciences 31(2), 1985.

Dataset: topic_datasets.zip / topic_datasets/cricket_deliveries.csv (synthetic data, 60,000 rows).